Software Re-Engineering

Fall 2026

Lecture 2


COMSATS INSTITUTE OF INFORMATION TECHNOLOGY
COMSATS
LAHORE


COMSATS
UNIVERSITY

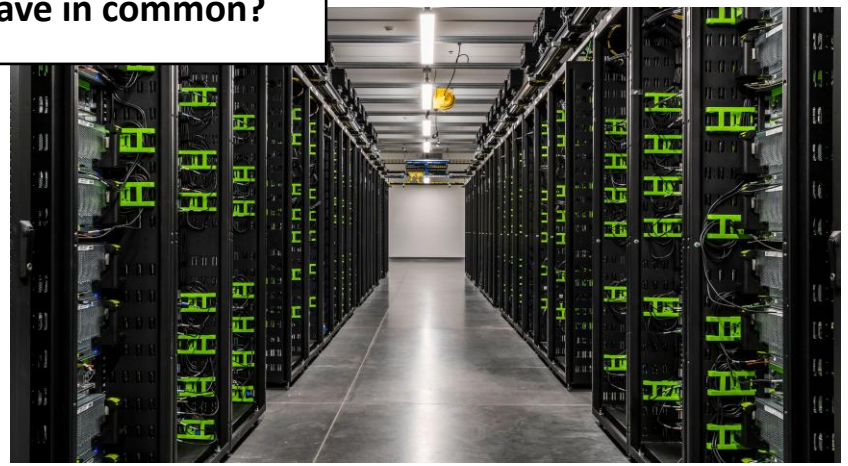






What do they all have in common?





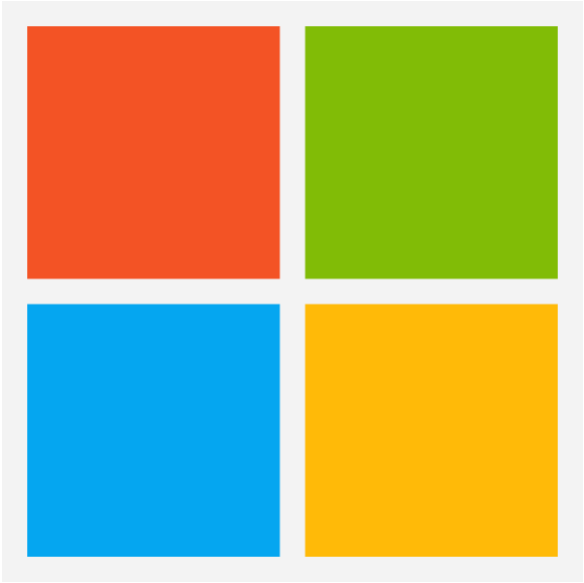
Software

- Software refers to computer programs, documentation, and associated data that enable computers and electronic devices to perform specific tasks, incorporating algorithms, user interfaces, and development tools.
  - **Saudi Arabia:** Annual oil export revenue was approximately $214 billion in 2025.
  - **India:** Annual IT/software export revenue reached approximately $246 billion in FY2025–26.
  - **Pakistan:** Annual IT & IT-enabled services export revenue reached approximately $3.8 billion in FY2024–25.






COMSATS UNIVERSITY
ISLAMABAD





Software Engineering vs Software Re-engineering

- **Software engineering**
  - The discipline of designing and building software systems from scratch.
- **Software re-engineering**
  - The process of improving and restructuring existing software systems to enhance their quality, performance, maintainability, and other attributes.
  - It involves analyzing, understanding, and modifying existing software systems to meet new requirements, address issues, or adapt to changes in technology.

When to re-engineer

- High technical debt
- Performance issues
- Scalability needs
- Security vulnerabilities
- Difficulty in adding new features
- Compliance with new standards
- Increased maintenance costs
- Integration with new technologies
- User dissatisfaction

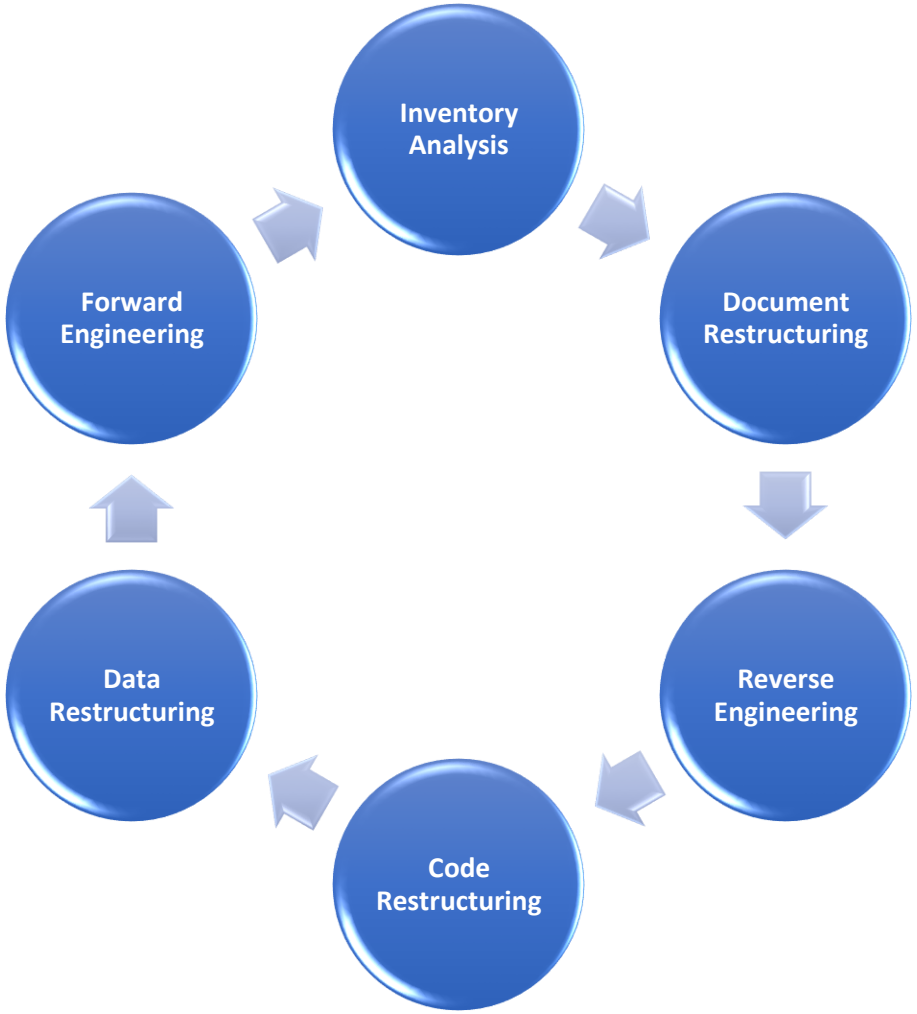
Importance of Re-engineering

1. **Enhances Software Maintainability and Scalability**
   - **Maintainability:** Software re-engineering improves the structure and organization of the code, making it easier for developers to understand, modify, and extend.
   - **Scalability:** By re-engineering the software, you can refactor the code and system architecture to handle increased workloads or expanded number of users.
2. **Reduces Technical Debt and System Complexity**
   - **Technical Debt:** Technical debt refers to the accumulation of poor design, code shortcuts, and other suboptimal decisions made during software development.
   - **System Complexity:** Over time, software systems can become increasingly complex due to the addition of new features, integration with other systems, and changes in business requirements.

Importance of Re-engineering

3. **Cost-Effective Compared to Developing New Software from Scratch**
   - Developing entirely new software can be time-consuming and expensive. This approach saves both time and money by avoiding the need to start from scratch. Additionally, re-engineering can be done incrementally, allowing businesses to continue using the system while improvements are made.
4. **Helps in Adapting to New Technologies and Business Requirements**
   - **Adapting to New Technologies:** The technology landscape is constantly evolving, and businesses need to keep up with the latest advancements to remain competitive.
   - **Meeting Business Requirements:** As businesses grow and evolve, their needs change.

Software Re-engineering Process



Inventory Analysis

- It is a process to assess the existing software assets within an organization. It involves taking details of all the software systems currently in use, evaluating their performance, and determining which systems are outdated, inefficient, or no longer align with the organization's goals.
  - **Identification:** Identify all software systems currently in use.
  - **Evaluation:** Assess each system's functionality, performance, and alignment with current business needs.
  - **Prioritization:** Rank systems based on the need for re-engineering, considering factors like cost, risk, potential benefits, and resource availability.
  - **Decision Making:** Decide which systems to re-engineer, retire, or replace.
- A prioritized list of systems or components that require re-engineering, along with a roadmap for addressing them.

Document Restructuring

- Over time, software documentation can become outdated, incomplete, or inconsistent with the actual code and system behavior. Document restructuring aims to bring documentation up to date, making it an accurate reflection of the current system. This step is crucial because clear, accurate documentation helps developers, maintainers, and stakeholders understand the system better.
  - **Review:** Review existing documentation for accuracy and completeness.
  - **Update:** Update documentation to reflect the current state of the system, including code comments, system design documents, user manuals, and technical specifications.
  - **Reorganization:** Reorganize documentation to improve readability and usability, ensuring it is easy to navigate and understand.
  - **Standardization:** Apply consistent standards and formats across all documentation.
- Comprehensive, accurate, and well-organized documentation that matches the current software system and supports ongoing maintenance and future development.

Reverse Engineering

- It Involves analyzing and understanding existing software to extract knowledge or recreate documentation when the source code or design is unavailable.
- Begins with an existing software system or source code.
- Focuses on understanding the internal workings of the system by analyzing its source code or behavior.
- Aims to understand, analyze, or modify an existing system.
- Used to recover lost code, analyze competitors' products, or migrate legacy systems.
- Involves decompilers, debuggers, and code analysis tools.

Code Restructuring

- It involves modifying the existing code to improve its internal structure without changing its external behavior. The goal is to make the code more readable, maintainable, and efficient, reducing technical debt and facilitating easier future modifications.
  - **Refactoring:** Improve code readability and structure by simplifying complex code, renaming variables and functions for clarity, and breaking down large functions or classes into smaller, more manageable pieces.
  - **Optimization:** Enhance code performance by optimizing algorithms, reducing redundancy, and improving resource management.
  - **Modularity:** Increase the modularity of the code, making it easier to reuse and maintain.
  - **Error Handling:** Improve error handling and exception management to make the code more robust.
- A cleaner, more maintainable source code that is easier to understand and modify, leading to more efficient future development and maintenance.

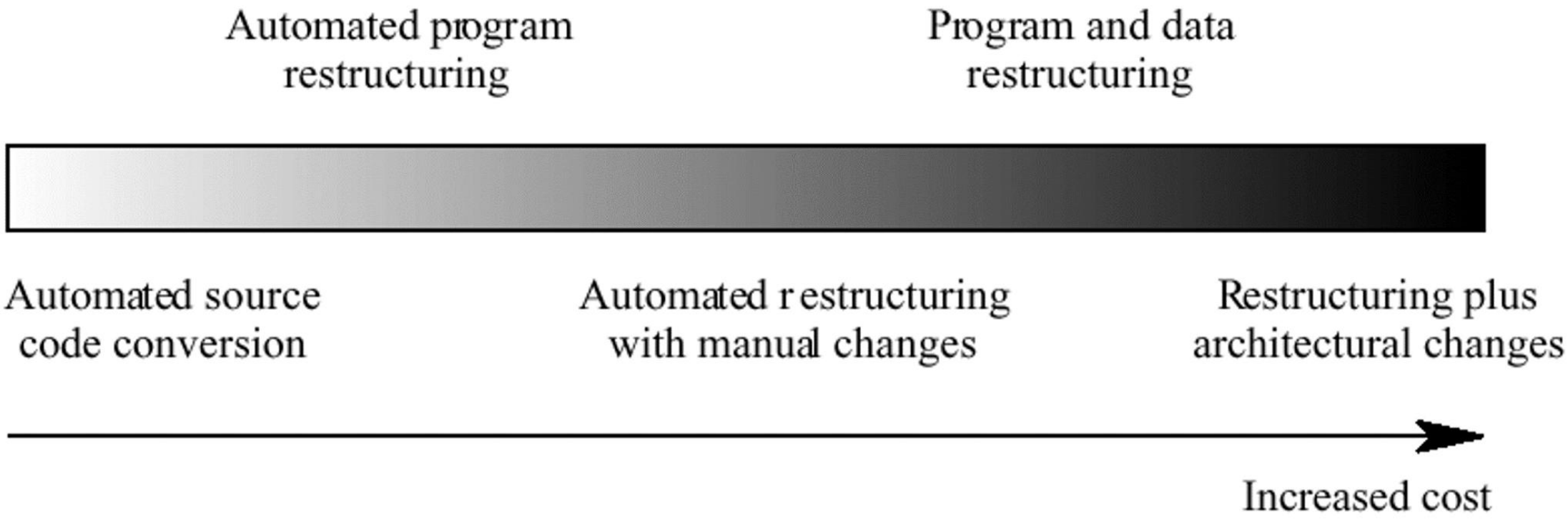
Data Restructuring

- It focuses on improving the organization, management, and storage of the software's data. This step is essential for enhancing the efficiency, reliability, and scalability of data operations within the system.
  - **Data Model Optimization:** Redesign data models to improve performance and reduce complexity. This might involve normalizing databases, optimizing indexes, or restructuring data tables.
  - **Data Migration:** Transfer data to a new structure or system, ensuring data integrity and consistency throughout the process.
  - **Data Cleaning:** Identify and remove inconsistencies, redundancies, and errors in the data to improve its quality.
  - **Integration:** Ensure that restructured data integrates seamlessly with other systems and applications.
- A more efficient, reliable, and scalable data structure that enhances overall system performance and supports future growth.

Forward Engineering

- Involves designing and developing software from specifications or requirements. It is the traditional process of creating software.
- Begins with requirements, design documents, and specifications.
- Follows a systematic approach of coding, testing, and implementation based on a pre-defined plan.
- Aims to create a new system or software from scratch.
- Involves programming languages, design tools, and compilers.

Re-engineering approaches



Automated Source Code Conversion

- This approach involves using automated tools to convert the source code from one programming language or platform to another.
- The goal is to modernize the code without altering its logic or structure significantly.
- **Cost:** This is typically the least expensive option since it relies heavily on automation and involves minimal manual intervention.

Automated Program Restructuring

- This approach goes a step further by automatically restructuring the code to improve its internal organization and maintainability.
- This may include refactoring code to enhance readability, efficiency, or modularity while preserving the original functionality.
- **Cost:** The cost is moderate, as it involves more complex automated processes than simple code conversion.

Automated Restructuring with Manual Changes

- In this approach, automated restructuring tools are used in conjunction with manual adjustments.
- While the automated tools handle the bulk of the restructuring, developers manually tweak the code to address issues that automation cannot handle, such as optimizing performance, fixing bugs, or adapting to specific business logic.
- **Cost:** This approach is more costly than pure automation due to the added manual work.

Program and Data Restructuring

- This approach involves restructuring not just the code but also the underlying data models and databases.
- It may include redesigning how data is stored, accessed, and managed to improve overall system performance and scalability.
- **Cost:** The cost is relatively high because it involves significant changes to both the program and data structures.

Restructuring Plus Architectural Changes

- This is the most extensive and costly approach, involving not only restructuring the code and data but also making significant changes to the system's architecture.
- This might include transitioning to a new software architecture style (e.g., from monolithic to microservices), incorporating new technologies, or completely redesigning how different components of the system interact.
- **Cost:** This approach has the highest cost because it involves a complete overhaul of the system.



Questions?